Software Requirements Specification

Restaurant App with Server and Database

SRS_v2 | Assignment 2 | Fall 2026

React Native (Expo) + Node.js / Express + Oracle Database

Repository: restaurant-app-mvp

Prepared by: [Your name] Roll No: [Your roll number]

Version 2.0 | October 2026

Revision History

Version	Date	Changes
1.0	A1	Original SRS. The app runs on mock data stored inside the phone.
2.0	Oct 2026	Backend added to scope. New terms (1.3). Database design replaced (Section 5). API endpoints added (Section 8). Sequence and deployment diagrams added (Section 9). Section 3 and Section 6 updated to use the server.

1. Introduction

1.1 Purpose

This document describes the requirements of the Restaurant App after Assignment 2. In A1 the app worked completely on mock data inside each phone. In A2 a Node.js and Express server and an Oracle database are added, so that every phone reads and writes the same data.

1.2 Scope

In scope

- Customer and manager sign up, log in and log out, with passwords stored as hashes and access controlled by a JWT token.
- Browsing the menu with category filter and search; the manager adds, edits, deletes items and switches availability.
- Placing orders. The server calculates the total from database prices and refuses unavailable items.
- Order tracking: the customer's app asks the server for the status every 10 seconds.
- Table reservations with a server check for free table, date, time and enough seats.
- Manager dashboard for menu, orders and reservations.
- The backend (Node.js, Express) and the database (Oracle) are now in scope. The assignment text uses MongoDB; Oracle is used here because the course allows any database. A collection corresponds to a table.

Out of scope

- Real online payments.
- Push notifications.
- Hosting on a public cloud server (the server runs on the developer's laptop on the same Wi-Fi as the phones).

Problems found in the A1 app and how A2 solves them

#	Problem in A1	How A2 solves it
1	The manager changes a price on his phone, but customers still see the old price.	Menu data lives in the database. Every phone loads it with GET /api/menu, and pull to refresh shows the new price.
2	When a dish runs out, it is marked unavailable only on the manager's phone.	is_available is stored in the menu_items table. The manager sets it once with PUT /api/menu/:id and all customers see it.
3	Orders and bookings are lost when the app is reinstalled, and are not visible on another phone.	Orders and reservations are saved in the database with the user id. GET /api/orders/my and GET /api/reservations/my return them after any login.
4	Two phones can book the same table at the same time, because each phone checks only its own data.	The server checks table, date, time and seats before saving. A clash returns 409.
5	Order status changes by a fake timer, not by the manager.	The manager changes the status with PATCH /api/orders/:id/status. The customer's app polls the server every 10 seconds.

#	Problem in A1	How A2 solves it
6	Users and passwords are kept in a mock file on the phone in readable form.	Users are stored in the database. Passwords are hashed with bcrypt and login returns a JWT that expires in one day.
7	The order total could be wrong or changed, because the app calculates it.	The server ignores any total sent by the app and calculates it from the prices in the database.
8	Any user could use the manager functions inside the app.	Add, edit and delete menu items, and all manager endpoints, need a valid token and the manager role (401 or 403 otherwise).

1.3 Definitions

Terms added in A2 (the A1 terms stay as they were):

Term	Meaning
API	Application Programming Interface. The set of URLs the server offers so that the app can read and change data.
REST	A style of API where URLs name things (for example /api/menu) and HTTP methods say what to do: GET reads, POST creates, PUT or PATCH changes, DELETE removes.
Endpoint	One URL together with one HTTP method, for example GET /api/menu/5.
JSON	JavaScript Object Notation. The text format used for request and response bodies.
HTTP Status Code	A number in every response that tells the result: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Server Error.
Node.js / Express	Node.js runs JavaScript on the server. Express is the library that handles routes and middleware.
Middleware	A function that runs before the controller, for example express.json(), cors(), protect and managerOnly.
JWT	JSON Web Token. A signed token given at login. The app sends it in the Authorization header so the server knows who the user is.
bcrypt	A one-way hashing method used to store passwords. The real password cannot be read back from the hash.
Oracle Database	The relational database used to store users, menu, tables, reservations and orders. Version 21c Express Edition.
Table, Row, Column	Oracle's names for what MongoDB calls collection, document and field.
Primary key / Foreign key	A primary key (id) identifies a row. A foreign key (for example orders.user_id) stores the id of a row in another table.
oracledb	The Node.js driver that connects the Express server to Oracle.
CORS	Cross-Origin Resource Sharing. The cors() middleware allows the app to call the server from another address.
Seeding	Filling the database with starting data (users, menu items, tables) by running npm run seed.
Polling	Asking the server for data again and again at a fixed time. Order Tracking polls every 10 seconds.
Environment variable	A setting kept in the .env file (PORT, JWT_SECRET, database user and password) and not pushed to GitHub.
BASE_URL	The server address used by the app, set in one place (src/api/client.js), for example http://192.168.1.15:5000.

2. Overall Description

2.1 Product perspective

The system has three parts: the React Native app on the phones, the Express server on the laptop, and the Oracle database. The app never talks to the database directly. It sends HTTP requests with JSON to the server, and the server runs the SQL (see the deployment diagram in Section 9).

2.2 User classes

User	What the user can do
Customer	Register and log in, view menu, place orders, track orders, book tables, view own orders and reservations.
Manager	Everything a customer can do, plus: add, edit, delete menu items, mark items available or unavailable, view all orders and change their status, accept or decline reservations.

2.3 Operating environment and constraints

- App: React Native with Expo Go on Android or iOS, or an emulator.
- Server: Node.js (LTS) with Express and the oracledb driver, running on the developer's laptop on port 5000.
- Database: Oracle Database 21c Express Edition on the laptop.
- The phone and the laptop must be on the same Wi-Fi. On the phone, localhost means the phone itself, so the laptop IP is used (Android emulator: 10.0.2.2).
- The laptop IP can change, so BASE_URL is kept in one file only.

3. Functional Requirements

ID	Requirement	Endpoints
FR-01	A new user can register with name, email and password. A duplicate email is refused with 409.	POST /api/auth/register
FR-02	A user can log in and receives a token (valid for one day) with name, email and role. The password is never returned.	POST /api/auth/login
FR-03	The app keeps the user logged in after it is closed. Logout clears the saved login.	(app side)
FR-04	Anyone can view the menu, filter it by category and search by name (not case sensitive).	GET /api/menu
FR-05	The manager can add, edit and delete menu items and switch availability.	POST, PUT, DELETE /api/menu
FR-06	A menu item that is part of an order cannot be deleted (409). The manager marks it unavailable instead.	DELETE /api/menu/:id
FR-07	A logged-in customer can place an order. The server calculates the total and rejects unavailable items.	POST /api/orders
FR-08	A customer can see only his own orders. The manager can see all orders.	GET /api/orders/my, GET /api/orders
FR-09	The manager moves an order forward: Pending, Preparing, Ready, Served. Cancelled is allowed only from Pending.	PATCH /api/orders/:id/status
FR-10	The Order Tracking screen refreshes the status from the server every 10 seconds.	GET /api/orders/my
FR-11	A logged-in customer can book a table. The server checks the table is free at that date and time and has enough seats (409 if booked).	POST /api/reservations
FR-12	A customer sees his own reservations. The manager sees all and can accept or decline.	GET /api/reservations/my, GET /api/reservations, PATCH /api/reservations/:id

ID	Requirement	Endpoints
FR-13	Manager-only endpoints return 401 without a token and 403 for a customer token.	all manager routes
FR-14	Every screen that loads data shows a loading indicator, an error message with a Retry button, and supports pull to refresh.	(app side)

4. Non-Functional Requirements

Area	Requirement
Security	Passwords are hashed with bcrypt. Tokens are signed with JWT_SECRET and expire in one day. .env and the database password are never pushed to GitHub. SQL uses bind variables, so user input cannot change the query.
Reliability	A wrong id, broken JSON or a database error returns a JSON error and does not crash the server. Unknown URLs return 404 "Route not found".
Performance	Normal requests answer within about one second on the local network. Order status is polled every 10 seconds.
Usability	The A1 screens and validation stay. The app shows clear loading and error states.
Maintainability	Routes only map URLs. Database code is in controllers. Server settings are in .env. BASE_URL is in one file.

5. Database Design

The A1 mock data sets are now tables in an Oracle database. In the assignment these are called collections. Ids are numbers generated by Oracle (identity columns). Column names use lowercase with underscores; the server converts them to camelCase JSON (for example is_available becomes isAvailable).

Table (Collection)	Fields and Types	Relationships and Rules
users	id NUMBER, primary key name VARCHAR2(100), required email VARCHAR2(150), required, unique password_hash VARCHAR2(100), required role VARCHAR2(10), customer or manager, default customer created_at TIMESTAMP, default now	The password is stored only as a bcrypt hash and is never sent in a response. The unique email gives the 409 error on register. One user can have many orders and many reservations.
menu_items	id NUMBER, primary key name VARCHAR2(100), required description VARCHAR2(500) price NUMBER(8,2), required, not negative category VARCHAR2(20), required: Starters, Mains, Desserts or Drinks image VARCHAR2(50), emoji or file name is_special NUMBER(1), 0 or 1, default 0 is_available NUMBER(1), 0 or 1, default 1	Referred to by order_items.menu_item_id. A used item cannot be deleted (Oracle error ORA-02292 becomes 409). Use is_available = 0 instead. Seed data: 16 items (4 Starters, 6 Mains, 3 Desserts, 3 Drinks).
restaurant_tables	id NUMBER, primary key table_number NUMBER, required, unique seats NUMBER(2), required, more than 0	Referred to by reservations.table_id and orders.table_id. Seed data: 6 tables.

Table (Collection)	Fields and Types	Relationships and Rules
reservations	id NUMBER, primary key user_id NUMBER, required, foreign key to users(id) table_id NUMBER, required, foreign key to restaurant_tables(id) reservation_date DATE, required reservation_time VARCHAR2(5), HH:MM, required guests NUMBER(2), required, more than 0 status VARCHAR2(10): Pending, Accepted or Declined, default Pending notes VARCHAR2(300) created_at, updated_at TIMESTAMP	Each reservation stores the id of the user who booked and of the table. Rule checked by the server: guests must not exceed seats, and no other Pending or Accepted reservation may exist for the same table, date and time (otherwise 409). Only the manager changes the status.
orders	id NUMBER, primary key user_id NUMBER, required, foreign key to users(id) table_id NUMBER, optional, foreign key to restaurant_tables(id) status VARCHAR2(10): Pending, Preparing, Ready, Served or Cancelled, default Pending total NUMBER(10,2), required, calculated by the server note VARCHAR2(300) created_at, updated_at TIMESTAMP	Each order stores the id of the user who placed it. Status moves only forward (Pending, Preparing, Ready, Served). Cancelled only from Pending. One order has one or more order_items.
order_items	id NUMBER, primary key order_id NUMBER, required, foreign key to orders(id), delete cascade menu_item_id NUMBER, required, foreign key to menu_items(id) quantity NUMBER(3), required, more than 0 unit_price NUMBER(8,2), required	This table replaces the array of items that a MongoDB order would hold inside the document. unit_price keeps the price at the time of the order, so a later price change does not change old orders.

*Collection to table: collection = table, document = row, Mongoose model = table script plus a data-access file, ObjectId reference = foreign key, Compass or Atlas = SQL*Plus or SQL Developer.*

6. User Interface and Data Sources

The A1 screens do not change. Only the source of the data changes, from mock files to the server.

Screen	A1 data source	A2 data source
Login and Signup	Mock users file	POST /api/auth/login, POST /api/auth/register; token and user saved in AuthContext and AsyncStorage
Menu	Mock menu with fake Promise	GET /api/menu (useApi hook), pull to refresh, Retry button
Cart and Place Order	Local state	POST /api/orders
Order Tracking	Fake timer	GET /api/orders/my every 10 seconds (setInterval, cleared on exit)
Reservations and My Reservations	Local storage	GET /api/tables, POST /api/reservations, GET /api/reservations/my
Manager Dashboard: Menu	Local state	POST, PUT, DELETE /api/menu, then reload the list
Manager Dashboard: Orders	Local state	GET /api/orders, PATCH /api/orders/:id/status
Manager Dashboard: Reservations	Local state	GET /api/reservations, PATCH /api/reservations/:id

7. Verification

- Postman collection restaurant-api.postman_collection.json: 12 requests for the menu API, including 5 that must fail with 400 or 404.
- Postman checks for authorization: no token gives 401, customer token gives 403, manager token gives 201.
- Two-device test: the manager changes a price and an order status on one device, and the customer sees both changes on the other device without restarting the app. A screen recording of at most four minutes is linked in README.md.

8. API Endpoints

8.1 Conventions

- Base URL: http://<laptop IP>:5000. All bodies and responses are JSON.
- Protected endpoints need the header Authorization: Bearer <token>, with one space after Bearer.
- Errors are returned as { "message": "..." } with a suitable status code.
- Access: Public = no token. Logged-in = any valid token. Manager = valid token and role manager.

8.2 Endpoint table

Method	Endpoint	Access	Request Body	Response
GET	`/api/health`	Public	None	{ status: "ok", time }
POST	`/api/auth/register`	Public	name, email, password	201 token and user; 409 if email exists
POST	`/api/auth/login`	Public	email, password	token and user (name, email, role); 401 if wrong
GET	`/api/menu`	Public	None. Optional query: category, search	List of menu items
GET	`/api/menu/:id`	Public	None	One item; 400 bad id; 404 not found
POST	`/api/menu`	Manager	name, price, category; optional description, image, isSpecial, isAvailable	201 and the new item; 400 invalid data
PUT	`/api/menu/:id`	Manager	Any of the item fields	Updated item; 400; 404
DELETE	`/api/menu/:id`	Manager	None	Message; 404; 409 if used by an order
GET	`/api/tables`	Logged-in	None	List of tables with seats
POST	`/api/orders`	Logged-in	items [{ menuItemId, quantity }], optional tableId, note	201 and the order with server-calculated total; 400 if an item is unavailable
GET	`/api/orders/my`	Logged-in	None	The user's own orders with items and status
GET	`/api/orders`	Manager	None	All orders
PATCH	`/api/orders/:id/status`	Manager	status	Updated order; 400 if the change is not allowed
POST	`/api/reservations`	Logged-in	tableId, date, time, guests, optional notes	201 and the reservation; 400 invalid or too many guests; 409 table already booked
GET	`/api/reservations/my`	Logged-in	None	The user's own reservations
GET	`/api/reservations`	Manager	None	All reservations
PATCH	`/api/reservations/:id`	Manager	status (Accepted or Declined)	Updated reservation; 400; 404

Total: 17 endpoints.

8.3 Status codes used

Code	When
200	Request succeeded (read, update, delete).
201	A row was created (register, menu item, order, reservation).
400	Missing or wrong data, invalid id, or a status change that is not allowed.
401	No token, or the token is wrong or expired; also wrong email or password at login.

Code	When
403	The user is logged in but is not a manager.
404	No such item, order or reservation, or the URL does not exist ("Route not found").
409	Email already registered, table already booked, or a menu item that is used by an order cannot be deleted.
500	Unexpected server or database error.

8.4 Business rules enforced by the server

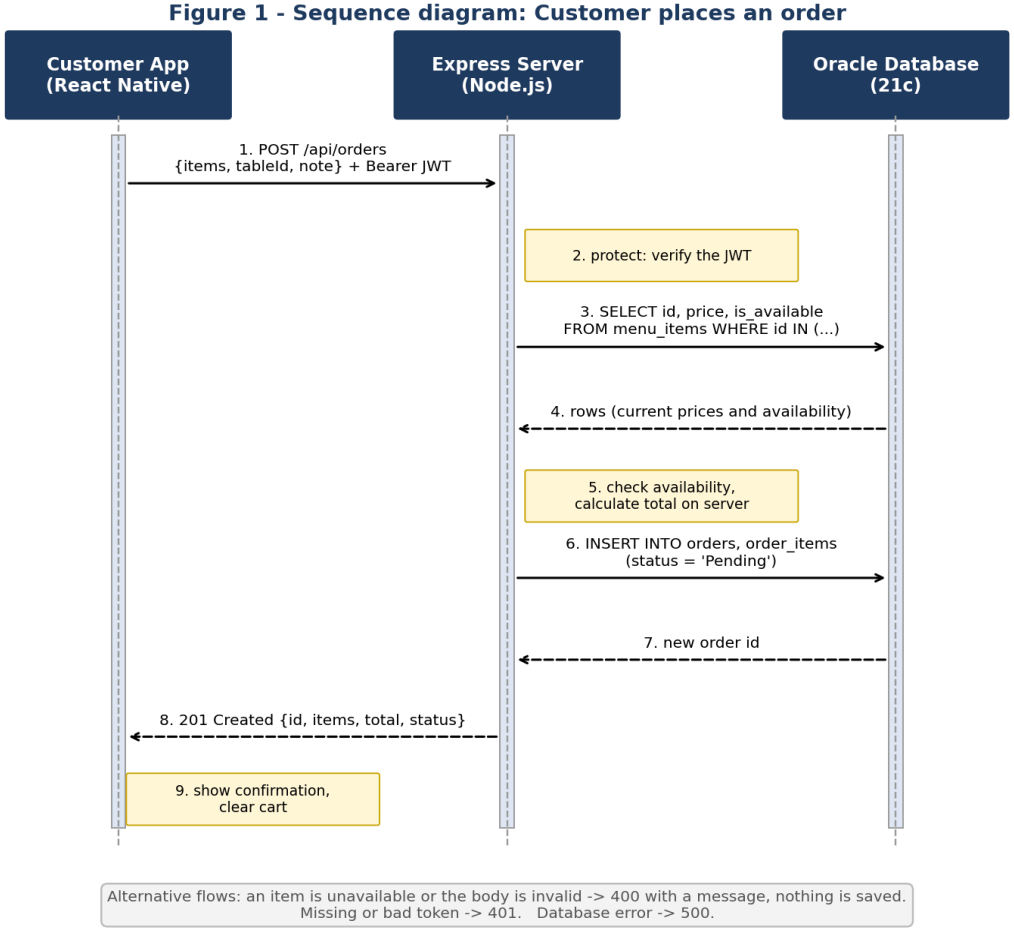
- The order total is calculated from the prices in the database. The total sent by the app is ignored, because a phone can be modified or can hold an old price.
- An order may contain only items with is_available = 1.
- Order status: Pending, then Preparing, then Ready, then Served. Cancelled is allowed only while the order is Pending. Any other change returns 400.
- A reservation is saved only if the table has enough seats and no other Pending or Accepted reservation exists for that table, date and time.
- A customer can read only his own orders and reservations.

9. Diagrams

9.1 Sequence diagram: Customer places an order

The A1 diagram is redrawn with three lifelines: the app, the Express server and the database. The app sends the request with its token, the server checks the token, reads the real prices and availability, calculates the total, saves the order and sends back the response.

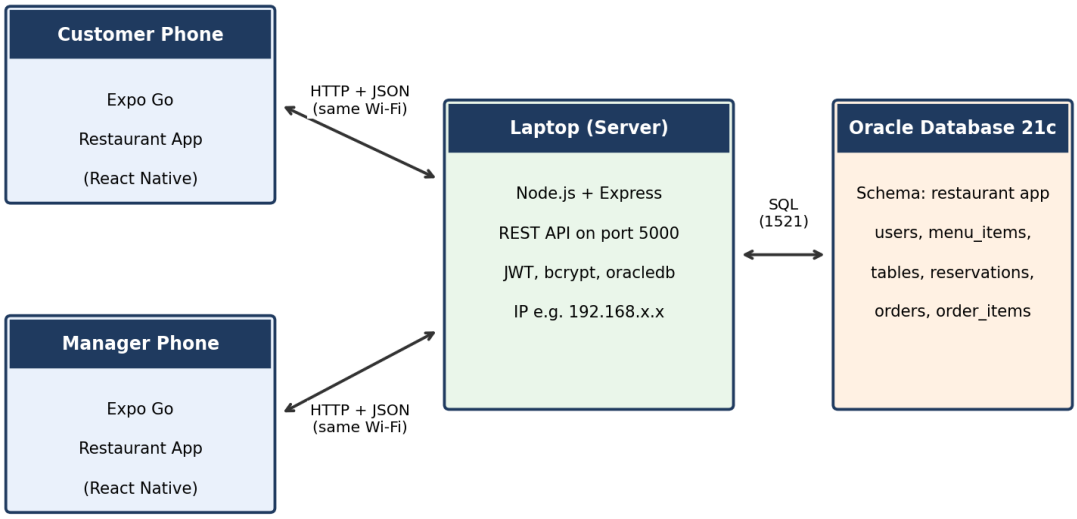
Figure 1 - Sequence diagram: Customer places an order



9.2 Deployment diagram

Both phones connect over Wi-Fi to the Express server on the laptop. Only the server connects to the Oracle database.

Figure 2 - Deployment diagram



Both phones use BASE_URL = http://<laptop IP>:5000 (Android emulator: http://10.0.2.2:5000)